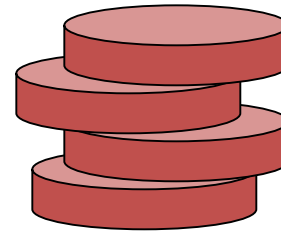
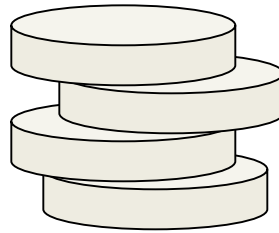
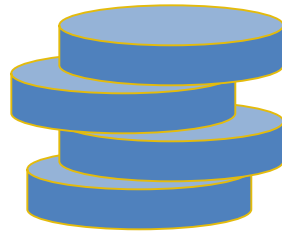
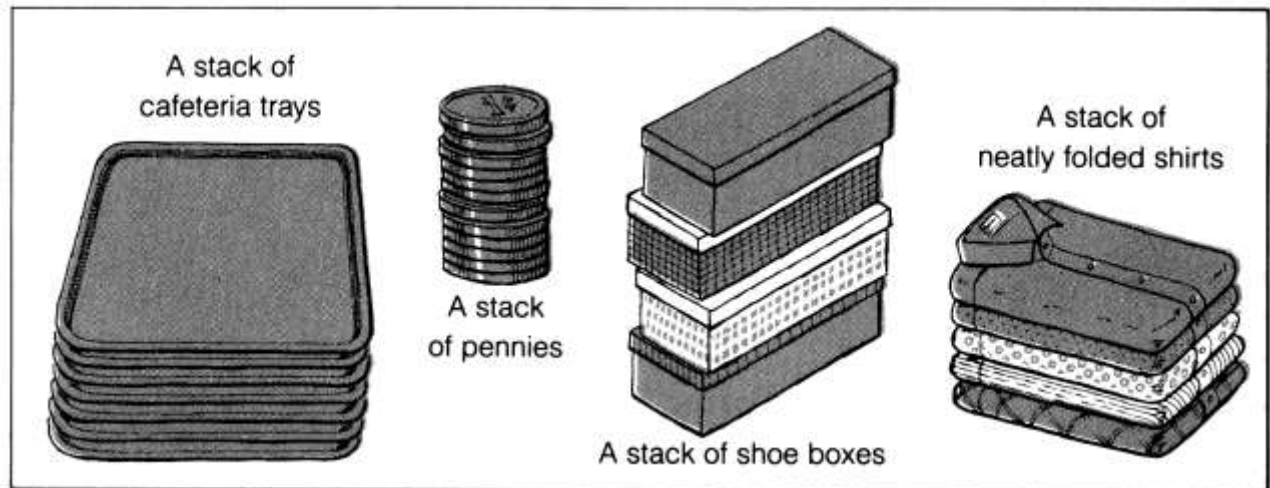


Stacks

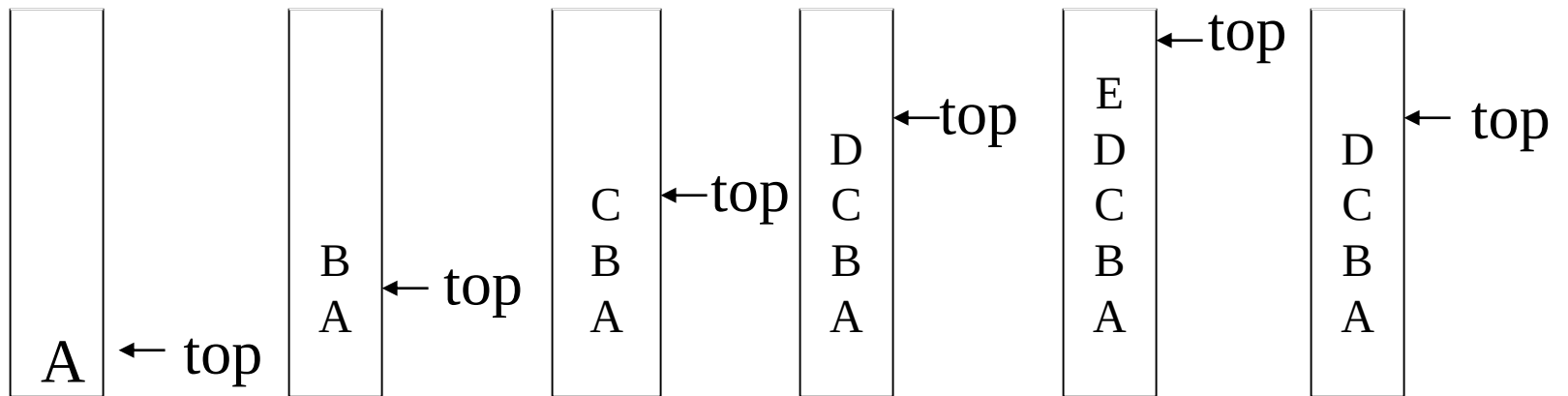


What is a Stack?

- ◆ Stores a set of elements in a particular order
- ◆ It is an ordered group of homogeneous items or elements
- ◆ Elements are added to and removed from the top of the stack
- ◆ Stack principle: **LAST IN FIRST OUT = LIFO**
- ◆ It means: the last element inserted is the first one to be removed
- ◆ Example



Last In First Out



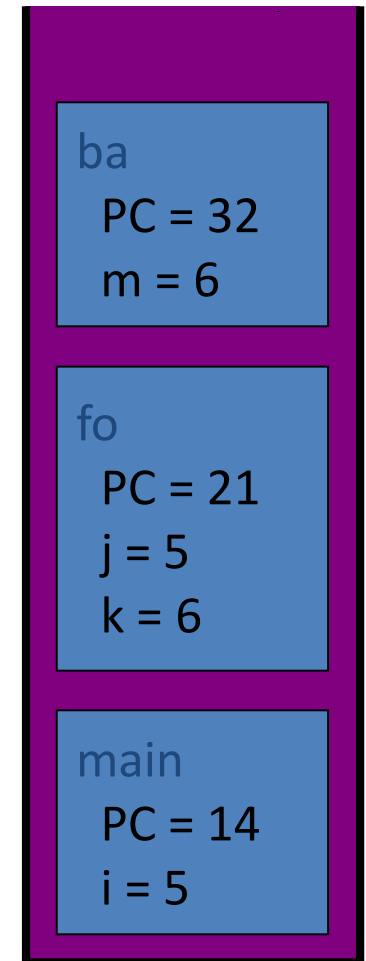
Applications of Stacks

- ▶ Direct applications
 - ▶ Page-visited history in a Web browser
 - ▶ Undo sequence in a text editor
 - ▶ Saving local variables when one function calls another, and this one calls another, and so on.
- ▶ Indirect applications
 - ▶ Auxiliary data structure for algorithms
 - ▶ Component of other data structures

C++ Run-time Stack

- ▶ The C++ run-time system keeps track of the chain of active functions with a stack
- ▶ When a function is called, the run-time system pushes on the stack a frame containing
 - ▶ Local variables and return value
 - ▶ Program counter, keeping track of the statement being executed
- ▶ When a function returns, its frame is popped from the stack and control is passed to the function on top of the stack

```
main() {  
    int i = 5;  
14  fo(i);  
}  
  
fo(int j) {  
    int k;  
    k = j+1;  
21  ba(k);  
}  
  
32 ba(int m) {  
    ...  
}
```



Array-based Stack

- ▶ A simple way of implementing the Stack uses an array
- ▶ We add elements from left to right
- ▶ A variable keeps track of the index of the top element

Algorithm *size()*

return $t + 1$

Algorithm *pop()*

if *isEmpty()* then

throw *EmptyStackException*

else

$t \leftarrow t - 1$

return $S[t + 1]$



Array-based Stack (cont.)

- ▶ The array storing the stack elements may become full
- ▶ A push operation will then throw a **FullStackException**

```
Algorithm push(x)  
  if  $t = S.length - 1$  then  
    throw FullStackException  
  else  
     $t \leftarrow t + 1$   
     $S[t] \leftarrow x$ 
```



Performance and Limitations

- Performance
 - Let n be the number of elements in the stack
 - The space used is $O(n)$
 - Each operation runs in time $O(1)$
- Limitations
 - The maximum size of the stack must be defined *a priori* , and cannot be changed
 - Trying to push a new element into a full stack causes an implementation-specific exception

Abstract Data Types (ADTs)

- ⇒ An abstract data type is a type and a collection of operations to manipulate that type
- ⇒ An abstract data type (ADT) is an abstraction of a data structure
- ⇒ An ADT specifies:
 - ⇒ Data stored
 - ⇒ Operations on the data
 - ⇒ Error conditions associated with operations
- ▶ Example: ADT modeling a simple stock trading system
 - ▶ The data stored are buy/sell orders
 - ▶ The operations supported are
 - ▶ order **buy**(stock, shares, price)
 - ▶ order **sell**(stock, shares, price)
 - ▶ void **cancel**(order)
 - ▶ Error conditions:
 - ▶ Buy/sell a nonexistent stock
 - ▶ Cancel a nonexistent order

The Stack ADT

- The **Stack** ADT stores arbitrary objects
 - Insertions and deletions follow the last-in first-out scheme
 - Main stack operations:
 - **push**(object e): inserts element e
 - **pop**(): removes and returns the last inserted element
- ▶ Auxiliary stack operations:
- ▶ **top**(): returns a reference to the last inserted element without removing it
 - ▶ **size**(): returns the number of elements stored
 - ▶ **isEmpty**(): returns a Boolean value indicating whether no elements are stored

Exceptions

- Attempting the execution of an operation of ADT may sometimes cause an error condition, called an exception
- Exceptions are said to be “thrown” by an operation that cannot be executed
- In the **Stack** ADT, operations **pop** and **top** cannot be performed if the stack is empty
- Attempting the execution of **pop** or **top** on an empty stack throws an **EmptyStackException**

List-based Stack Implementation: Push

```
void push(int item)
{
    if (top >= MAX_SIZE - 1) {
        cout << "Stack Overflow: Cannot push " << val << ". Stack
is full." << endl;
    } else {
        top++;
        stack_arr[top] = val;
        cout << val << " pushed to stack." << endl;
    }
}
```

List-based Stack Implementation: Pop

```
element pop(pnode top) {  
    /* delete an element from the stack */  
    if (top < 0) {  
        cout << "Stack Underflow: Cannot pop. Stack is empty."  
        << endl;  
    } else {  
        cout << "Popped element: " << stack_arr[top] << endl;  
        top--;  
    }  
}
```